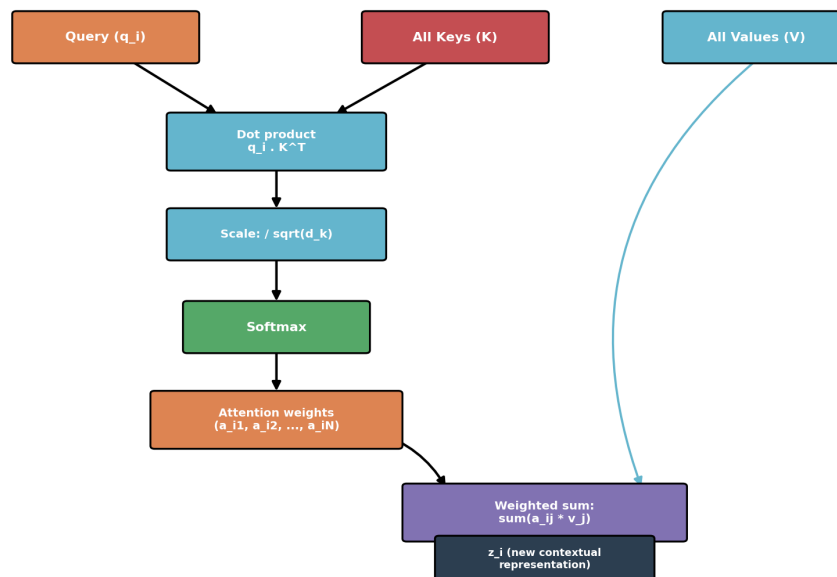


Introduction to Transformers

Part 1: Impact & Origins | Part 2: Self-Attention

How Self-Attention Works - Full Mechanism



Part 1: Why transformers were created • Impact on NLP • Democratizing & accelerating AI • Unifying deep learning • Timeline • Advantages/disadvantages • The future

Part 2: What is self-attention • The 'average meaning' problem • How it works • Parallel operation • Progressive summarization • The attention matrix • Building Q/K/V vectors

PART 1 - Introduction to Transformers

1. Why Were Transformers Created?

Before 2017, state-of-the-art sequence models (RNNs, LSTMs, GRUs, even attention-augmented Seq2Seq models) all shared one core limitation: they processed sequences **step by step**. Each time step depended on the one before it, which meant training was inherently slow and hard to parallelize - especially on modern GPUs, which are built for massive parallel computation.

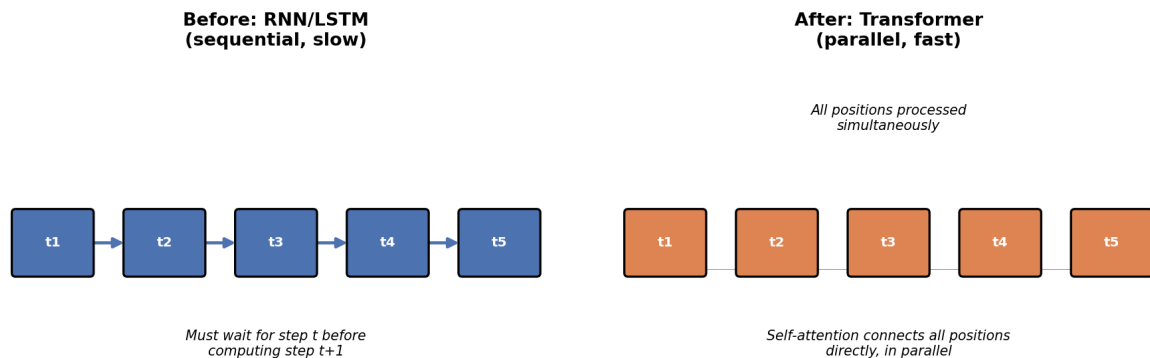


Figure 1: RNNs process sequentially and can't be parallelized; the Transformer's self-attention connects all positions at once.

The 2017 paper "*Attention Is All You Need*" asked: what if we removed recurrence completely, and relied purely on attention to model relationships between words - regardless of their distance in the sequence? The result, the **Transformer**, could be trained almost entirely in parallel, unlocking dramatically faster training on much larger datasets.

2. Timeline of the Transformer Era

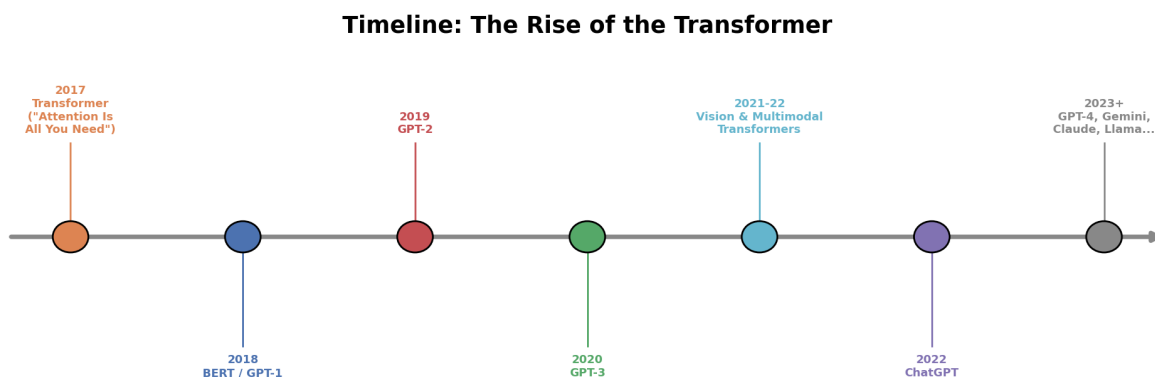


Figure 2: From the original 2017 paper to today's frontier models - all built on the same core idea.

3. Impact of Transformers on NLP

Transformers didn't just improve NLP benchmarks incrementally - they reset what was considered possible. Tasks like translation, summarization, question answering, and text generation all saw large jumps in quality within just a couple of years of the architecture's introduction.

- Enabled training on **web-scale** text data, since training could be parallelized efficiently.
- Made a **single pretrained model** reusable across many downstream tasks via fine-tuning (building on the transfer learning paradigm).
- Led directly to breakthrough models: BERT, GPT, T5, and eventually ChatGPT and other modern LLMs.

4. Democratizing AI

Because pretrained Transformer models could be shared and fine-tuned cheaply, powerful NLP capabilities became accessible to far more people - not just large labs with massive compute budgets. Open-source model hubs, APIs, and community fine-tuning made state-of-the-art AI usable by individual developers, students, and small companies.

5. Acceleration of AI

The efficiency gains from parallel training, combined with the architecture's ability to keep improving as it scales, triggered a rapid acceleration in AI capability and adoption after 2017.

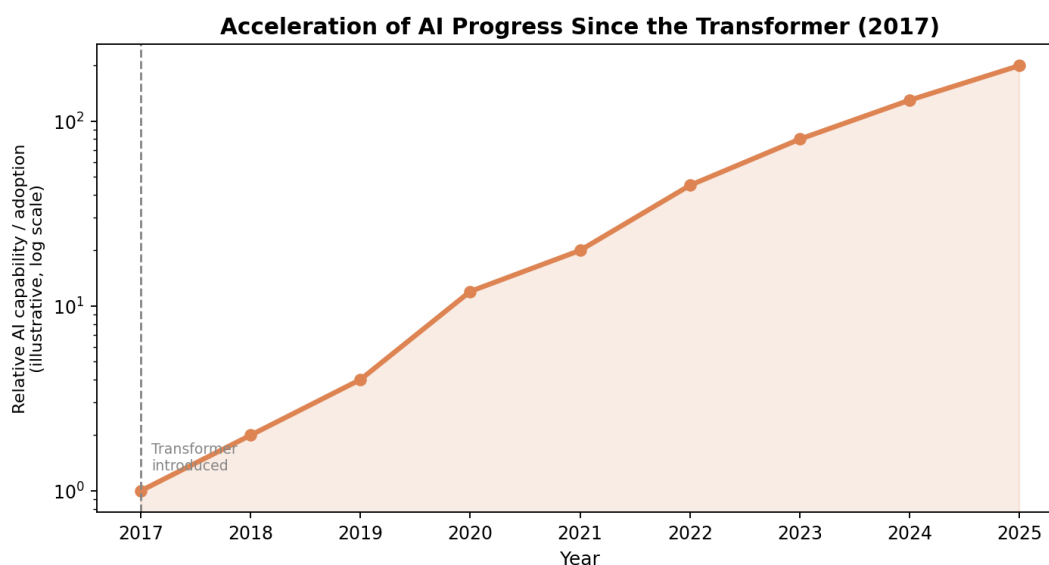


Figure 3: Illustrative growth in AI capability/adoption since the Transformer's introduction (log scale).

6. Unification of Deep Learning

Perhaps the most surprising impact: the Transformer turned out not to be NLP-specific at all. The same core architecture - embeddings plus self-attention plus feed-forward layers - has since been successfully applied to images, audio, video, protein structures, and more, unifying what used to be separate sub-fields of deep learning under one dominant architecture.

Unification: One Architecture, Many Domains

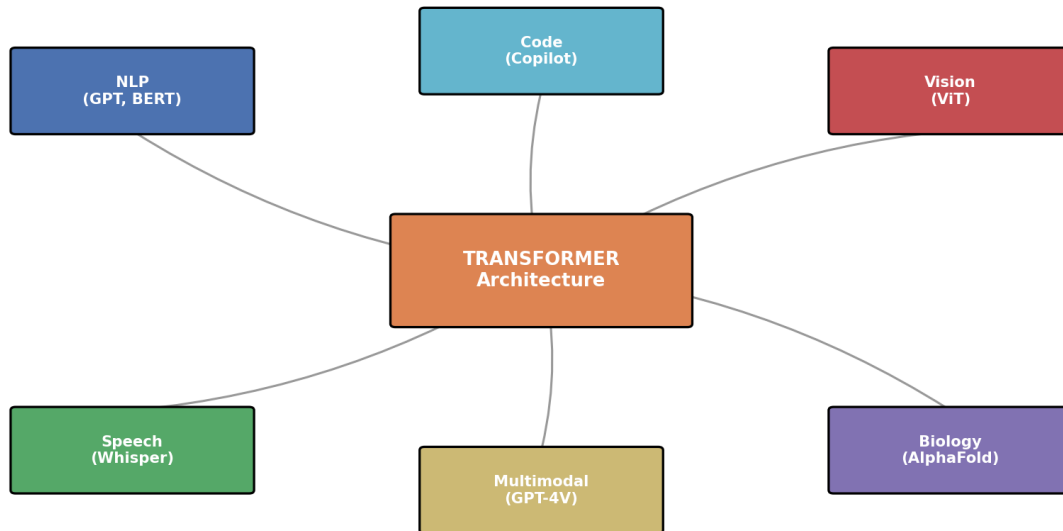


Figure 4: The same Transformer architecture now powers language, vision, speech, biology, and multimodal models.

7. Advantages & Disadvantages

Transformers - Advantages vs Disadvantages

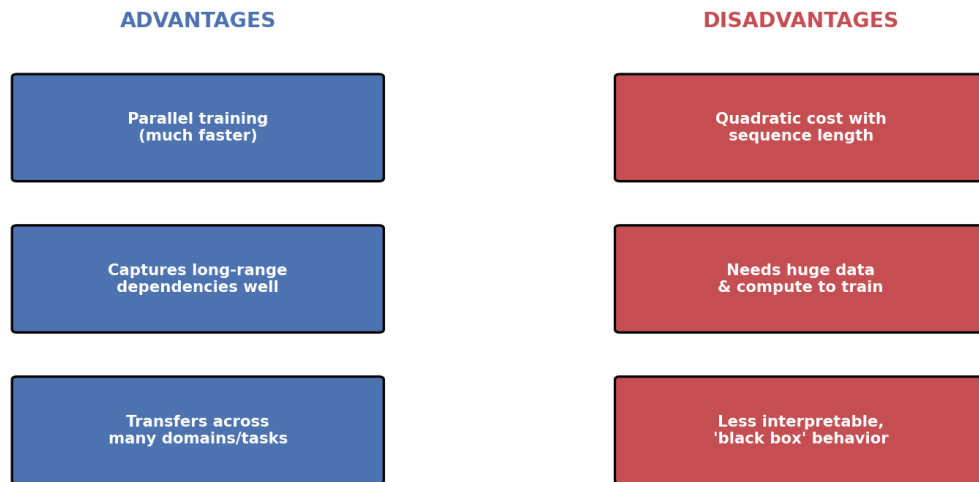


Figure 5: Key trade-offs of the Transformer architecture.

- **Advantage:** parallel training makes far better use of modern hardware than sequential RNNs.
- **Advantage:** self-attention directly connects any two positions, capturing long-range dependencies without the decay problem that limits RNNs.
- **Disadvantage:** self-attention's compute and memory cost grows **quadratically** with sequence length, making very long inputs expensive.
- **Disadvantage:** Transformers are data-hungry and compute-hungry to train from scratch, and their internal reasoning remains difficult to fully interpret.

8. The Future of Transformers

Where Are Transformers Headed?

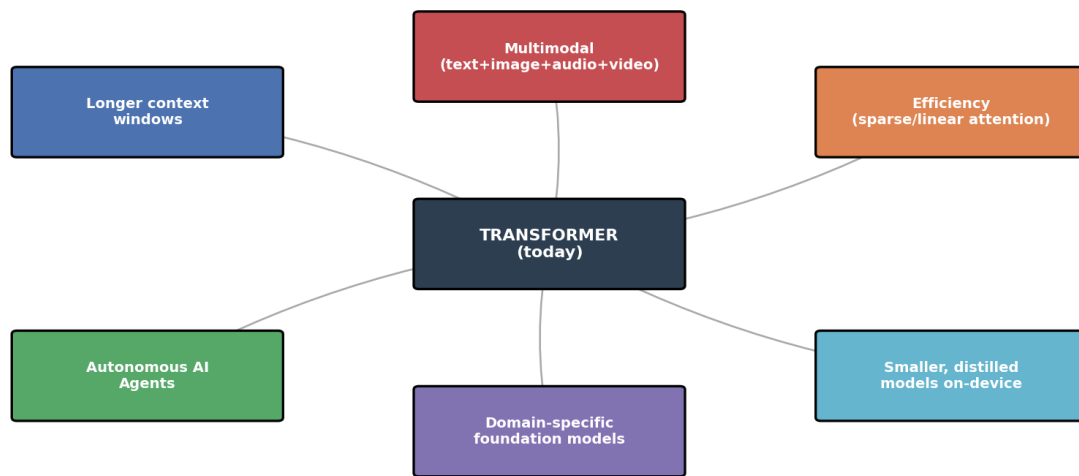


Figure 6: Active research directions building on the Transformer foundation.

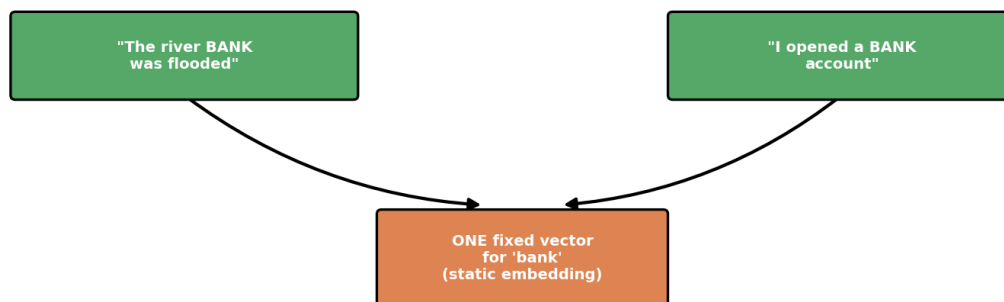
Active areas of development include longer context windows, more efficient attention variants (to reduce the quadratic cost), deeper multimodal integration, smaller distilled models for on-device use, and Transformer-based autonomous agents that can plan and take actions, not just generate text.

PART 2 - What Is Self-Attention?

9. The Problem with "Average Meaning"

Older word embedding methods (like Word2Vec or GloVe) assign each word exactly **one** fixed vector, no matter the sentence it appears in. But many words have multiple meanings depending on context - a single static vector can only represent something like an "average" of all those meanings, which fits none of them well.

The Problem with "Average Meaning" (Static Embeddings)



A single static embedding must represent BOTH meanings at once - it ends up as an 'average' of all senses, capturing neither well

Figure 7: A static embedding for 'bank' has to blend the riverbank meaning and the financial meaning into one vector.

This is exactly the problem self-attention was designed to solve: instead of a fixed vector per word, self-attention produces a **context-aware** vector for each word, built dynamically from the specific sentence it appears in.

Self-Attention: Context-Aware Meaning



Self-attention lets each word's vector be re-weighted using its surrounding context, producing a DIFFERENT representation of 'bank' in each sentence

Figure 8: With self-attention, 'bank' gets a DIFFERENT vector in each sentence, shaped by its actual context.

10. Building Query, Key & Value Vectors from Embeddings

Self-attention starts from each word's input embedding x , and transforms it into **three** different vectors using three separate learned weight matrices: W_q , W_k , and W_v .

Building Query, Key & Value Vectors from Embeddings

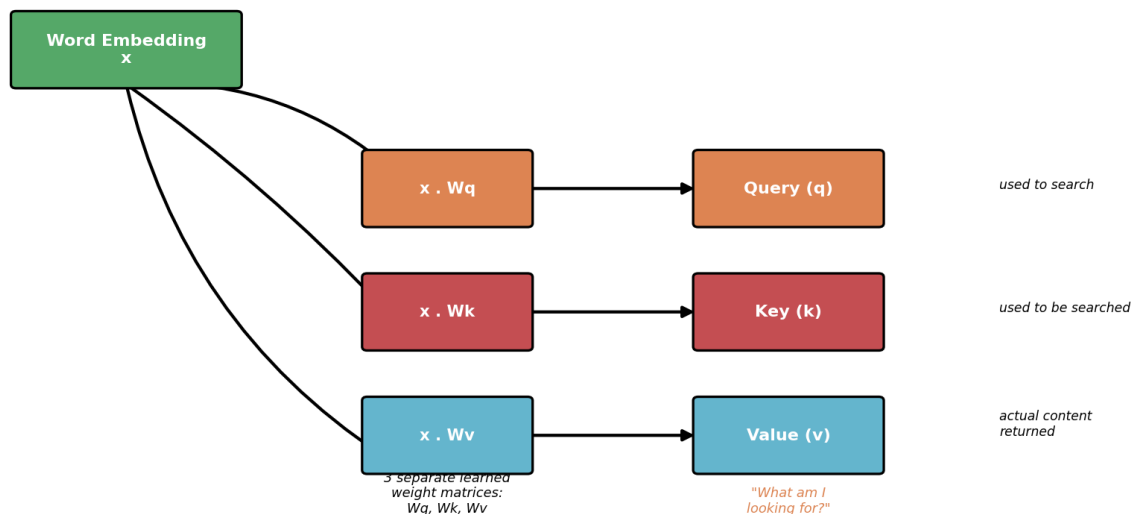


Figure 9: Every word embedding is projected into a Query, a Key, and a Value vector using three learned weight matrices.

- **Query (q)** - represents what this word is 'looking for' in the rest of the sentence.
- **Key (k)** - represents what this word 'offers' or advertises to others looking for it.
- **Value (v)** - the actual content/information this word contributes once it's been attended to.

$$q(i) = x(i) \cdot W_q \quad k(i) = x(i) \cdot W_k \quad v(i) = x(i) \cdot W_v$$

11. How Does Self-Attention Work?

For every word, self-attention compares its Query against every word's Key to figure out how relevant each other word is, then uses those relevance scores to blend together everyone's Value vectors.

How Self-Attention Works - Full Mechanism

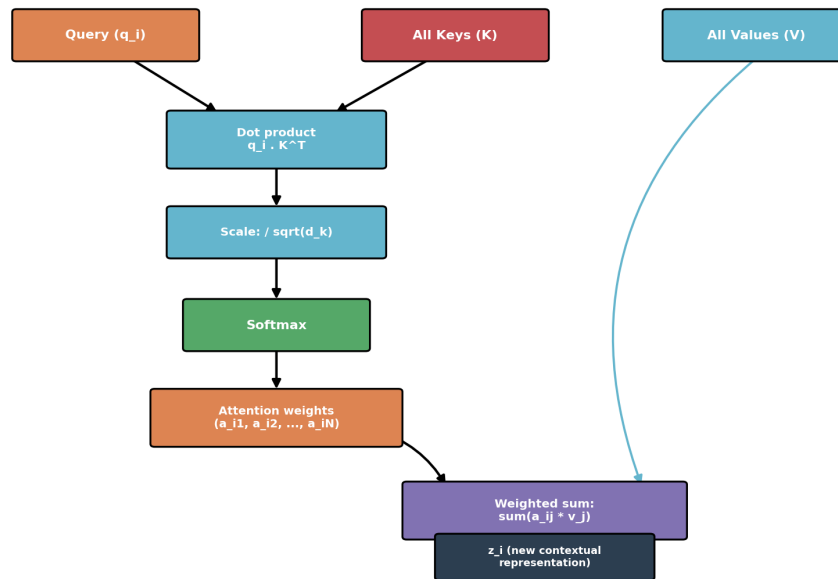


Figure 10: The full self-attention mechanism for one word - dot product, scale, softmax, then weighted sum of Values.

- **1. Score:** compute the dot product between this word's Query and every word's Key.
- **2. Scale:** divide by $\sqrt{d_k}$ to keep the values numerically stable for softmax.
- **3. Softmax:** convert scores into attention weights that sum to 1.
- **4. Weighted sum:** multiply each Value vector by its attention weight and sum them up, producing this word's new, context-aware representation $z(i)$.

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

12. The Summarized Attention Matrix

For a full sentence, this process happens for every word at once, producing an $N \times N$ **attention matrix** (N = number of words), where each row shows how much that word attends to every other word (including itself). Each row always sums to 1, since it's the output of a softmax.

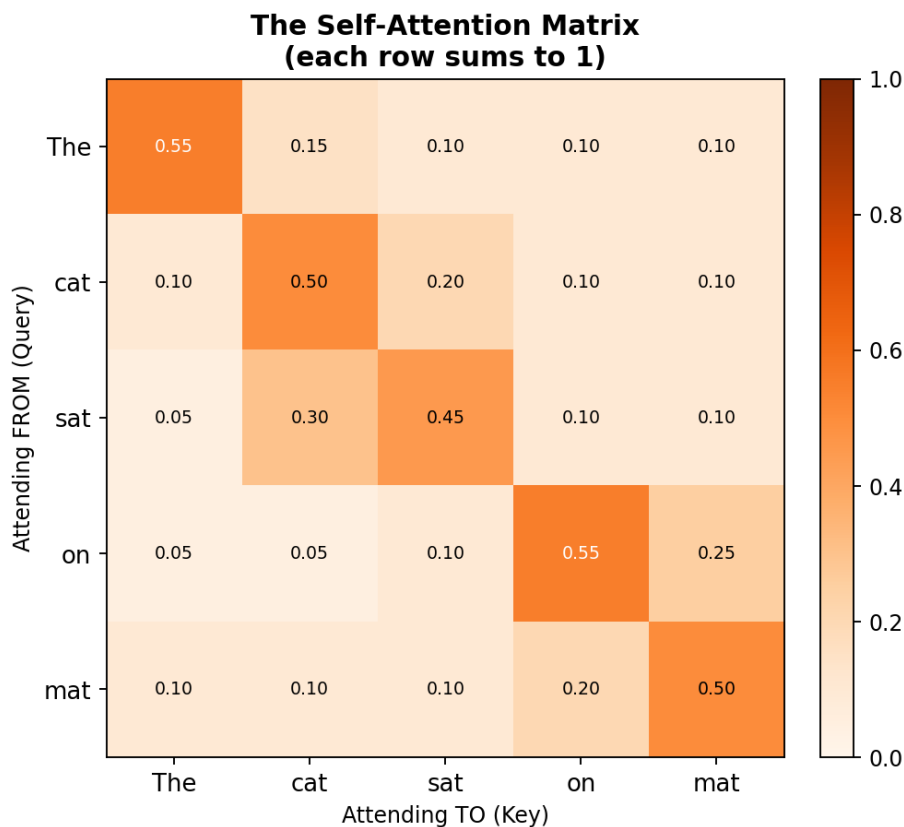


Figure 11: A self-attention matrix - brighter cells mean stronger attention between that pair of words.

13. Parallel Operation

Because Q , K , and V for ALL words can be computed via simple matrix multiplication (not a loop over time steps), self-attention for an entire sentence is computed in one shot - this is the core reason Transformers train so much faster than RNNs.

Parallel Operation: Everything Is Matrix Multiplication

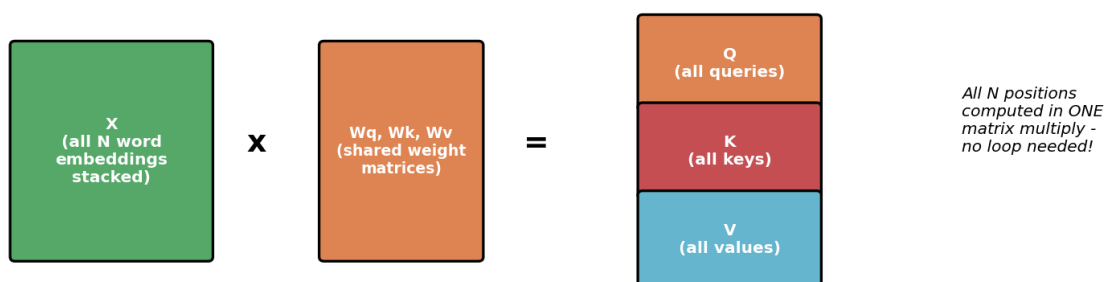


Figure 12: Stacking all word embeddings into one matrix X lets Q , K , V for the WHOLE sentence be computed in a single matrix multiply.

14. Progressive Summarization

A single self-attention layer only captures one 'round' of context-mixing. Real Transformers stack many such layers on top of each other. With each additional layer, every word's representation is refined again using the FULL sentence, so meaning becomes progressively richer - from raw word identity, to local context, to full sentence-level understanding.

Progressive Summarization: Meaning Builds Up Layer by Layer

Each layer's self-attention refines every word's vector using the FULL sentence, so meaning becomes progressively richer and more context-aware with depth

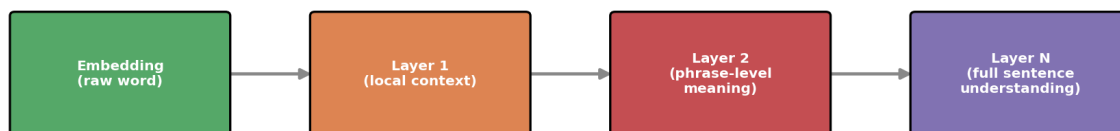


Figure 13: Each Transformer layer progressively summarizes more context into every word's vector representation.

Summary

Concept	Key Idea
Why Transformers were created	To remove sequential bottlenecks and enable parallel training
Impact	Reset NLP benchmarks, unified deep learning across domains, accelerated AI progress
Advantages	Parallelizable, strong long-range dependency modeling, broadly transferable
Disadvantages	Quadratic compute cost, data/compute hungry, limited interpretability
The 'average meaning' problem	Static embeddings can't represent multiple word senses at once
Self-attention's fix	Builds a context-aware vector for each word, dynamically, per sentence
Q, K, V	Query = what I seek; Key = what I offer; Value = what I contribute
Core formula	$\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$
Why it's fast	All positions processed via matrix multiplication - no sequential loop

Together, these two parts cover both the 'why' and the 'how' of Transformers: Part 1 showed why this architecture reshaped AI, and Part 2 broke down the self-attention mechanism that makes it all possible - the true engine at the heart of every modern LLM.